

```
// — BFS / DFS Traversal —

void bfs(int start, const vector<vector<int>> &adj, vector<bool> &visited) {
    queue<int> q;
    visited[start] = true;
    q.push(start);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : adj[u])
            if (!visited[v]) {
                visited[v] = true;
                q.push(v);
            }
    }
}

void dfs(int u, const vector<vector<int>> &adj, vector<bool> &visited) {
    visited[u] = true;
    for (int v : adj[u])
        if (!visited[v])
            dfs(v, adj, visited);
}

vector<int> bfsShortestPath(int start, int end,
                           const vector<vector<int>> &adj) {
    int n = adj.size();
    vector<int> dist(n, -1), parent(n, -1);
    queue<int> q;
    dist[start] = 0;
    q.push(start);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : adj[u])
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                parent[v] = u;
                q.push(v);
            }
    }
    if (dist[end] == -1)
        return {};
    vector<int> path;
    for (int cur = end; cur != -1; cur = parent[cur])
        path.push_back(cur);
    reverse(path.begin(), path.end());
    return path;
}
```

GRAFOS

```
// — Flood Fill —  
  
void floodFill(vector<vector<int>> &grid, int r, int c, int oldColor,  
              int newColor) {  
    if (r < 0 || r >= (int)grid.size() || c < 0 || c >= (int)grid[0].size())  
        return;  
    if (grid[r][c] != oldColor)  
        return;  
    grid[r][c] = newColor;  
    floodFill(grid, r + 1, c, oldColor, newColor);  
    floodFill(grid, r - 1, c, oldColor, newColor);  
    floodFill(grid, r, c + 1, oldColor, newColor);  
    floodFill(grid, r, c - 1, oldColor, newColor);  
}  
  
int countIslands(vector<vector<int>> &grid) {  
    int rows = grid.size(), cols = grid[0].size(), count = 0;  
    for (int i = 0; i < rows; i++)  
        for (int j = 0; j < cols; j++)  
            if (grid[i][j] == 1) {  
                floodFill(grid, i, j, 1, 0);  
                count++;  
            }  
    return count;  
}
```

GRAFOS

// — Floyd-Warshall —

```
const int INF = 1e9;
```

```
vector<vector<int>> floydWarshall(vector<vector<int>> dist) {  
    int n = dist.size();  
    for (int k = 0; k < n; k++)  
        for (int i = 0; i < n; i++)  
            for (int j = 0; j < n; j++)  
                if (dist[i][k] < INF && dist[k][j] < INF)  
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);  
    return dist;  
}
```

GRAFOS

```
// — MST: Kruskal —  
  
struct DSU {  
    vector<int> parent, rank;  
    DSU(int n) : parent(n), rank(n, 0) { iota(parent.begin(), parent.end(), 0); }  
    int find(int x) { return parent[x] == x ? x : parent[x] = find(parent[x]); }  
    bool unite(int x, int y) {  
        x = find(x);  
        y = find(y);  
        if (x == y)  
            return false;  
        if (rank[x] < rank[y])  
            swap(x, y);  
        parent[y] = x;  
        if (rank[x] == rank[y])  
            rank[x]++;  
        return true;  
    }  
};  
  
struct Edge {  
    int u, v, w;  
};  
  
long long kruskal(int n, vector<Edge> &edges) {  
    sort(edges.begin(), edges.end(),  
        [](const Edge &a, const Edge &b) { return a.w < b.w; });  
    DSU dsu(n);  
    long long cost = 0;  
    for (auto &e : edges)  
        if (dsu.unite(e.u, e.v))  
            cost += e.w;  
    return cost;  
}
```

GRAFOS

```
// — MST: Prim —  
  
long long prim(int n, const vector<vector<pair<int, int>>> &adj) {  
    vector<int> dist(n, INT_MAX);  
    vector<bool> inMST(n, false);  
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<>> pq;  
    dist[0] = 0;  
    pq.push({0, 0});  
    long long cost = 0;  
    while (!pq.empty()) {  
        auto [d, u] = pq.top();  
        pq.pop();  
        if (inMST[u])  
            continue;  
        inMST[u] = true;  
        cost += d;  
        for (auto [v, w] : adj[u])  
            if (!inMST[v] && w < dist[v]) {  
                dist[v] = w;  
                pq.push({w, v});  
            }  
    }  
    return cost;  
}
```

GRAFOS

```
// — Bipartite Matching (Hopcroft-Karp) —————

struct BipartiteMatching {
    int L, R;
    vector<vector<int>> adj;
    vector<int> matchL, matchR, dist;

    BipartiteMatching(int L, int R)
        : L(L), R(R), adj(L), matchL(L, -1), matchR(R, -1), dist(L) {}

    void addEdge(int u, int v) { adj[u].push_back(v); }

    bool bfs() {
        queue<int> q;
        for (int u = 0; u < L; u++)
            if (matchL[u] == -1) {
                dist[u] = 0;
                q.push(u);
            } else
                dist[u] = INT_MAX;
        bool found = false;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int v : adj[u]) {
                int w = matchR[v];
                if (w == -1)
                    found = true;
                else if (dist[w] == INT_MAX) {
                    dist[w] = dist[u] + 1;
                    q.push(w);
                }
            }
        }
        return found;
    }

    bool dfs(int u) {
        for (int v : adj[u]) {
            int w = matchR[v];
            if (w == -1 || (dist[w] == dist[u] + 1 && dfs(w))) {
                matchL[u] = v;
                matchR[v] = u;
                return true;
            }
        }
        dist[u] = INT_MAX;
        return false;
    }

    int maxMatching() {
        int matching = 0;
        while (bfs())
            for (int u = 0; u < L; u++)
                if (matchL[u] == -1 && dfs(u))
                    matching++;
        return matching;
    }
};
```